

Assignment #1 – Microservices Architecture

System: Food Delivery System

Team members: Alfa-Kilo, Sierra-Lima, Elephant-Yankee, Mike-Alfa

Part 1 – Realistic system

Overview. QuickBite, a platform that connects customers with nearby restaurants and independent delivery partners. Customers browse menus, place and pay for orders, and track deliveries; restaurants manage menus and order preparation; couriers accept jobs and update delivery progress in real time.

Who uses it.

- Customers (order food via web/mobile).
- Restaurants (manage menus, accept orders, update preparation status).
- Delivery partners (accept delivery jobs, navigate pickup -> drop-off, share location).
- Platform admins (monitor performance, resolve disputes, view analytics).

Main functionalities.

- User registration/login and profiles (customer/restaurant/courier/admin roles).
- Restaurant and menu discovery with search/filters (cuisine, distance, rating, price).
- Cart and order placement with item customisation.
- Payment processing and refunds.
- Real-time order and courier tracking on a map.
- Notifications (push/SMS/email) for each status change.
- Ratings and reviews for restaurants and couriers.
- Order history and basic restaurant performance statistics.

Why multiple microservices are necessary.

- Different business domains have different data ownership and workflows (users, menus, orders, payments, deliveries, notifications, reviews).
- Payments and refunds are security-sensitive and benefit from strict isolation (even if simulated now).
- Delivery tracking needs low-latency, high-frequency updates and scales differently from order transactions.
- Traffic spikes affect ordering and tracking differently; services must scale independently.
- Notifications must be resilient and non-blocking (fire-and-forget with retries).
- Independent deployment allows changing one domain (e.g., courier matching) without redeploying the entire system.

Therefore, the system cannot reasonably be built as a single monolith without creating tight coupling and conflicting non-functional requirements.

Part 2 – Architecture

This is a logical microservices architecture (business-level decomposition and integration).

Step 1 – Service ownership

Team member	Microservice A	Microservice B
Alfa-Kilo	User Service	Order Service
Sierra-Lima	Menu Service	Restaurant Service
Elephant-Yankee	Delivery Service	Payment Service
Mike-Alfa	Notification Service	Review Service

Step 2 – Microservices and responsibilities

1. User Service (Owner: Alfa-Kilo)

- Does: Provides authentication/authorisation and user profile management for all roles.
- Manages: Accounts, roles/permissions, profiles, addresses, courier verification data.
- Why separated: Security-critical, shared by all other services, and benefits from independent scaling and caching.

2. Order Service (Owner: Alfa-Kilo)

- Does: Owns the end-to-end order lifecycle (Placed -> Confirmed -> Prepared -> Ready -> Picked -> Delivered).
- Manages: Carts, order creation, order status/state machine, order history.
- Why separated: Highest transaction volume and coordination logic; must orchestrate several other services.

3. Menu Service (Owner: Sierra-Lima)

- Does: Publishes each restaurant's food catalogue for browsing and ordering.
- Manages: Menu items, categories, prices, customisations, availability, images.
- Why separated: Read-heavy and frequently updated by restaurants; separate ownership and scaling from orders/payments.

4. Restaurant Service (Owner: Sierra-Lima)

- Does: Onboards restaurants and maintains their public/business profile.
- Manages: Restaurant details, location, opening hours, open/closed status, approval workflow.
- Why separated: Different lifecycle and data sensitivity (partner/compliance data) from customer ordering.

5. Delivery Service (Owner: Elephant-Yankee)

- Does: Matches couriers to paid orders and tracks delivery progress.
- Manages: Courier availability (available/unavailable), assignment, delivery status and location updates.
- Why separated: Real-time updates and matching logic require low latency and separate scaling from transactional services.

6. Payment Service (Owner: Elephant-Yankee)

- Does: Processes payments for orders and handles refund flows (simulated).
- Manages: Payment attempts (success/failure), refund requests, payment status, transaction records.
- Why separated: Financial logic has its own lifecycle and failure handling; isolation reduces coupling and improves security posture.

7. Notification Service (Owner: Mike-Alfa)

- Does: Sends outbound communications to customers, restaurants and couriers.
- Manages: Message templates, channels (email/SMS/push), delivery retries and dead-lettering.
- Why separated: High-volume and asynchronous by nature; must not block core flows when external providers are slow.

8. Review Service (Owner: Mike-Alfa)

- Does: Collects ratings/reviews for restaurants and couriers after delivery.
- Manages: Review text, numeric ratings, aggregates (averages), moderation flags.
- Why separated: Read-heavy and analytics-friendly workload; can scale independently without impacting ordering.

Step 3 – Dependencies (who depends on whom and why)

Order Service (central coordinator):

- Calls User Service to validate the customer identity and delivery address.
- Calls Restaurant Service to check restaurant status (open/closed) and basic restaurant data.
- Calls Menu Service to validate ordered items, availability and prices.
- Calls Payment Service to authorise/capture payment or trigger refunds on cancellation.
- Calls Delivery Service to create a delivery job after successful payment.
- Calls Notification Service to inform participants about order status changes.

Delivery Service:

- Calls User Service to read courier profiles and availability.
- Exchanges status updates with Order Service (pickup/delivered) to progress the order lifecycle.
- Calls Notification Service for courier assignment and ETA updates.

Review Service:

- Calls Order Service to ensure a review can only be created after the order is Delivered.
- Calls User Service to link the review to the authenticated reviewer.

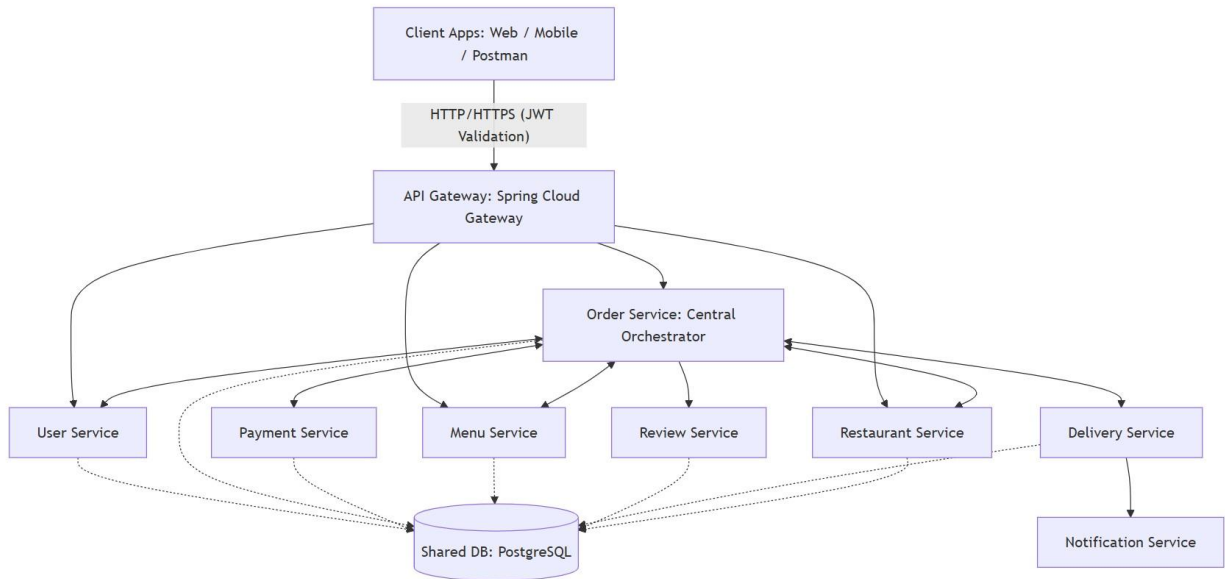
Menu Service:

- Calls Restaurant Service because menus are owned by a specific restaurant.

Cross-cutting dependencies:

- All services rely on User Service for authentication/authorisation (e.g., validating JWTs).
- Notification Service is invoked asynchronously by core services and must tolerate downstream outages.

Step 4 – Architecture diagram



Arrows indicate service-to-service dependencies (business-level calls/data needs).

Figure 1. Logical microservices architecture for the Food Delivery System.